

μOS++ III SMP

Test commands, one by one

μOS++ III SMP — test commands, one by one

Every QEMU, hardware and host test of the AArch32, AArch64, Cortex-M33, RP2350, STM32F4 and POSIX platforms, written as the explicit commands that the runners execute — `test_smpl/run-qemu.sh`, `test_smpl/run-hw.sh` and `test_smpl/run-host.sh`, each board's `hw.sh`, and the `add_test()` commands of the kernel platforms — without the scripts.

Platform	Port	QEMU tests	Hardware tests	Host tests	Section
<code>aarch32-rpi-zero-2w</code>	<code>micro-os-plus-iii-aarch32</code>	15	15	—	3.3, 4.3
<code>aarch32-rpi3b</code>	<code>micro-os-plus-iii-aarch32</code>	15	14	—	3.4, 4.4
<code>aarch64-rpi-zero-2w</code>	<code>micro-os-plus-iii-aarch64</code>	15	15	—	3.5, 4.5
<code>aarch64-rpi3b</code>	<code>micro-os-plus-iii-aarch64</code>	15	14	—	3.6, 4.6
<code>aarch32-luckfox-lyra</code>	<code>micro-os-plus-iii-aarch32</code>	—	20	—	4.7
<code>2xcortex-m33</code>	<code>micro-os-plus-iii-cortexm</code>	4	—	—	5.2
<code>pico2-1cpu</code>	<code>micro-os-plus-iii-cortexm</code>	4	—	—	5.3
<code>cortexm-pico2</code>	<code>micro-os-plus-iii-cortexm</code>	6	20	—	5.4, 6.3, 6.4
<code>cortexm-pico2-rp2350b-psram</code>	<code>micro-os-plus-iii-cortexm</code>	3	15	—	5.5, 6.5
<code>cortexm-pico2-pizero</code>	<code>micro-os-plus-iii-cortexm</code>	—	14	—	6.6
<code>cortexm-nucleof411</code>	<code>micro-os-plus-iii-cortexm</code>	—	4	—	7.2
<code>cortexm-weactf411</code>	<code>micro-os-plus-iii-cortexm</code>	—	5	—	7.3
<code>cortexm-weactf412</code>	<code>micro-os-plus-iii-cortexm</code>	—	5	—	7.4
<code>native</code> (<code>native-cmake-sys</code>)	<code>micro-os-plus-iii-posix-arch</code>	—	—	16	8

The image names and the test lists were read on 2026-10-09 from the **debug** builds (`*-cmake-gcc-debug` , `native-cmake-sys-debug`) made from:

Repository	Branch	Commit
<code>micro-os-plus-iii</code>	<code>smp</code>	<code>d780c76c</code>
<code>micro-os-plus-iii-aarch32</code>	<code>smp</code>	<code>e721dea</code>
<code>micro-os-plus-iii-aarch64</code>	<code>smp</code>	<code>8151c26</code>
<code>micro-os-plus-iii-cortexm</code>	<code>smp</code>	<code>71e98ff</code>
<code>micro-os-plus-iii-posix-arch</code>	<code>smp</code>	<code>4ed05ad</code>

1. Paths and tools

```
WORK=/tmp
K=$WORK/micro-os-plus-iii          # kernel
A32=$WORK/micro-os-plus-iii-aarch32 # AArch32 port
A64=$WORK/micro-os-plus-iii-aarch64 # AArch64 port
C=$WORK/micro-os-plus-iii-cortexm  # Cortex-M port
P=$WORK/micro-os-plus-iii-posix-arch # POSIX port
BUILD=$K/tests/build

QEMU=$HOME/.local/xPacks/@xpack-dev-tools/qemu-arm/9.2.4-1.1/.content/bin/qemu-system-aarch64
QEMUARM=$HOME/.local/xPacks/@xpack-dev-tools/qemu-arm/9.2.4-1.1/.content/bin/qemu-system-arm
OPENOCD=$HOME/.local/xPacks/@xpack-dev-tools/openocd/0.12.0-7.1/.content/bin/openocd
SCRIPTS=$HOME/.local/xPacks/@xpack-dev-tools/openocd/0.12.0-7.1/.content/openocd/scripts
TC32=$HOME/.local/xPacks/@xpack-dev-tools/arm-none-eabi-gcc/15.2.1-1.1.1/.content/bin
TC64=$HOME/.local/xPacks/@xpack-dev-tools/aarch64-none-elf-gcc/15.2.1-1.1.1/.content/bin
```

These are the versions the scripts pick on this machine: the newest `qemu-arm` and `openocd` xPacks installed, and the 15.x toolchains.

2. Install and build

```
cd $A32 && xpm install && xpm link
cd $A64 && xpm install && xpm link
cd $C    && xpm install && xpm link
cd $P    && xpm install && xpm link

cd $K/tests
for c in aarch32-rpi-zero-2w aarch32-rpi3b aarch64-rpi-zero-2w aarch64-rpi3b aarch32-luckfox-lyra \
        2xcortex-m33 pico2-1cpu cortexm-pico2 cortexm-pico2-rp2350b-psram cortexm-pico2-pizero \
        cortexm-nucleof411 cortexm-weactf411 cortexm-weactf412; do
    xpm install      --config $c-cmake-gcc-debug
    xpm run prepare  --config $c-cmake-gcc-debug
    xpm run build    --config $c-cmake-gcc-debug
done
xpm install      --config native-cmake-sys-debug
xpm run prepare  --config native-cmake-sys-debug
xpm run build    --config native-cmake-sys-debug
```

(Replace `-debug` by `-release` for the release images.) The AArch32 and AArch64 images are in `$BUILD/<platform>-cmake-gcc-debug/platform-bin/port-tests/test/`:

File	Used by
<code><test>-qemu.bin</code>	QEMU (raw image)
<code>shim8.img</code>	QEMU, AArch32 only (the AArch64-to-AArch32 boot shim)
<code><test>-hwd</code>	hardware (the ELF; a link to <code><test>-hwd.elf</code>)

3. QEMU

3.1 The command

AArch32 — QEMU's `raspi3b` starts its cores in AArch64, so it boots `shim8.img`, which drops to AArch32 and jumps to the test image loaded at `0x10000`:

```
timeout <seconds> $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    [-drive file=<card image>,if=sd,format=raw] \
    -kernel $D/shim8.img -device loader,file=$D/<test>-qemu.bin,addr=0x10000 \
    < /dev/null
```

AArch64 — the test image is the kernel:

```

timeout <seconds> $QEMU -M raspi3b -smp 4 -nographic -serial none \
  -semihosting-config enable=on,target=native \
  [-drive file=<card image>,if=sd,format=raw] \
  -kernel $D/<test>-qemu.bin \
  < /dev/null

```

The test passed when its output has a line `RESULT: PASS`. `RESULT: SKIP` is a skip (`usb_test`: QEMU has no USB device mode). `RESULT: FAIL`, no `RESULT` line, or exit status 124 (the `timeout` expired) is a failure.

3.2 The SD card image

Four tests need a card: a fresh image before the run, deleted after it.

```

# sd_test: a card seeded with a flatfs volume, by the test's own tool
python3 <port>/test/<board>/sd_test/flatfs_tool.py make-seed /tmp/sd_test.disk.img

# smp-mat-sdcard-test, smp-num-test, smp-pipeline-test: a blank 4 GiB sparse file
truncate -s 4G /tmp/<test>.disk.img

rm -f /tmp/<test>.disk.img      # after the run

```

Checked on 2026-10-09 with these exact commands: AArch32 and AArch64 `smp_test0` (`RESULT: PASS (10 heartbeats on core 0)`, exit 0), and AArch32 `sd_test` with a seeded card (`RESULT: PASS (SD + flatfs verified)`, exit 0).

3.3 `aarch32-rpi-zero-2w` — QEMU

```
D=$BUILD/aarch32-rpi-zero-2w-cmake-gcc-debug/platform-bin/port-tests/test
```

`cmsis-os-validator`

```

timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \
  -semihosting-config enable=on,target=native \
  -kernel $D/shim8.img -device loader,file=$D/cmsis-os-validator-qemu.bin,addr=0x10000 \
  < /dev/null

```

`mutex-stress`

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/mutex-stress-qemu.bin,addr=0x10000 \  
< /dev/null
```

rtos-apis

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/rtos-apis-qemu.bin,addr=0x10000 \  
< /dev/null
```

sd_test

```
python3 $A32/test/rpi-zero-2w/sd_test/flatfs_tool.py make-seed /tmp/sd_test.disk.img  
timeout 450 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-drive file=/tmp/sd_test.disk.img,if=sd,format=raw \  
-kernel $D/shim8.img -device loader,file=$D/sd_test-qemu.bin,addr=0x10000 \  
< /dev/null  
rm -f /tmp/sd_test.disk.img
```

smp-mat-sdcard-test

```
truncate -s 4G /tmp/smp-mat-sdcard-test.disk.img  
timeout 2000 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-drive file=/tmp/smp-mat-sdcard-test.disk.img,if=sd,format=raw \  
-kernel $D/shim8.img -device loader,file=$D/smp-mat-sdcard-test-qemu.bin,addr=0x10000 \  
< /dev/null  
rm -f /tmp/smp-mat-sdcard-test.disk.img
```

smp-mat-test

```
timeout 900 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/smp-mat-test-qemu.bin,addr=0x10000 \  
< /dev/null
```

smp-num-test

```
truncate -s 4G /tmp/smp-num-test.disk.img
timeout 1000 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -drive file=/tmp/smp-num-test.disk.img,if=sd,format=raw \
    -kernel $D/shim8.img -device loader,file=$D/smp-num-test-qemu.bin,addr=0x10000 \
    < /dev/null
rm -f /tmp/smp-num-test.disk.img
```

smp-pipeline-test

```
truncate -s 4G /tmp/smp-pipeline-test.disk.img
timeout 1000 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -drive file=/tmp/smp-pipeline-test.disk.img,if=sd,format=raw \
    -kernel $D/shim8.img -device loader,file=$D/smp-pipeline-test-qemu.bin,addr=0x10000 \
    < /dev/null
rm -f /tmp/smp-pipeline-test.disk.img
```

smp-pro-cons-test

```
timeout 1000 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -kernel $D/shim8.img -device loader,file=$D/smp-pro-cons-test-qemu.bin,addr=0x10000 \
    < /dev/null
```

smp_test0

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -kernel $D/shim8.img -device loader,file=$D/smp_test0-qemu.bin,addr=0x10000 \
    < /dev/null
```

smp_test1

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -kernel $D/shim8.img -device loader,file=$D/smp_test1-qemu.bin,addr=0x10000 \
    < /dev/null
```

smp_test2

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/smp_test2-qemu.bin,addr=0x10000 \  
< /dev/null
```

smp_test3

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/smp_test3-qemu.bin,addr=0x10000 \  
< /dev/null
```

smp_test4

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/smp_test4-qemu.bin,addr=0x10000 \  
< /dev/null
```

usb_test

```
timeout 200 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/usb_test-qemu.bin,addr=0x10000 \  
< /dev/null
```

3.4 aarch32-rpi3b — QEMU

```
D=$BUILD/aarch32-rpi3b-cmake-gcc-debug/platform-bin/port-tests/test
```

cmsis-os-validator

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/cmsis-os-validator-qemu.bin,addr=0x10000 \  
< /dev/null
```

mutex-stress

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/mutex-stress-qemu.bin,addr=0x10000 \  
< /dev/null
```

rtos-apis

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/rtos-apis-qemu.bin,addr=0x10000 \  
< /dev/null
```

sd_test

```
python3 $A32/test/rpi3b/sd_test/flatfs_tool.py make-seed /tmp/sd_test.disk.img  
timeout 450 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-drive file=/tmp/sd_test.disk.img,if=sd,format=raw \  
-kernel $D/shim8.img -device loader,file=$D/sd_test-qemu.bin,addr=0x10000 \  
< /dev/null  
rm -f /tmp/sd_test.disk.img
```

smp-mat-sdcard-test

```
truncate -s 4G /tmp/smp-mat-sdcard-test.disk.img  
timeout 2000 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-drive file=/tmp/smp-mat-sdcard-test.disk.img,if=sd,format=raw \  
-kernel $D/shim8.img -device loader,file=$D/smp-mat-sdcard-test-qemu.bin,addr=0x10000 \  
< /dev/null  
rm -f /tmp/smp-mat-sdcard-test.disk.img
```

smp-mat-test

```
timeout 900 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/smp-mat-test-qemu.bin,addr=0x10000 \  
< /dev/null
```

smp-num-test

```
truncate -s 4G /tmp/smp-num-test.disk.img
timeout 1000 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -drive file=/tmp/smp-num-test.disk.img,if=sd,format=raw \
    -kernel $D/shim8.img -device loader,file=$D/smp-num-test-qemu.bin,addr=0x10000 \
    < /dev/null
rm -f /tmp/smp-num-test.disk.img
```

smp-pipeline-test

```
truncate -s 4G /tmp/smp-pipeline-test.disk.img
timeout 1000 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -drive file=/tmp/smp-pipeline-test.disk.img,if=sd,format=raw \
    -kernel $D/shim8.img -device loader,file=$D/smp-pipeline-test-qemu.bin,addr=0x10000 \
    < /dev/null
rm -f /tmp/smp-pipeline-test.disk.img
```

smp-pro-cons-test

```
timeout 1000 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -kernel $D/shim8.img -device loader,file=$D/smp-pro-cons-test-qemu.bin,addr=0x10000 \
    < /dev/null
```

smp_test0

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -kernel $D/shim8.img -device loader,file=$D/smp_test0-qemu.bin,addr=0x10000 \
    < /dev/null
```

smp_test1

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -kernel $D/shim8.img -device loader,file=$D/smp_test1-qemu.bin,addr=0x10000 \
    < /dev/null
```

smp_test2

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/smp_test2-qemu.bin,addr=0x10000 \  
< /dev/null
```

smp_test3

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/smp_test3-qemu.bin,addr=0x10000 \  
< /dev/null
```

smp_test4

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/smp_test4-qemu.bin,addr=0x10000 \  
< /dev/null
```

usb_test

```
timeout 200 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/shim8.img -device loader,file=$D/usb_test-qemu.bin,addr=0x10000 \  
< /dev/null
```

3.5 aarch64-rpi-zero-2w — QEMU

```
D=$BUILD/aarch64-rpi-zero-2w-cmake-gcc-debug/platform-bin/port-tests/test
```

cmsis-os-validator

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/cmsis-os-validator-qemu.bin \  
< /dev/null
```

mutex-stress

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/mutex-stress-qemu.bin \  
< /dev/null
```

rtos-apis

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/rtos-apis-qemu.bin \  
< /dev/null
```

sd_test

```
python3 $A64/test/rpi-zero-2w/sd_test/flatfs_tool.py make-seed /tmp/sd_test.disk.img  
timeout 450 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-drive file=/tmp/sd_test.disk.img,if=sd,format=raw \  
-kernel $D/sd_test-qemu.bin \  
< /dev/null  
rm -f /tmp/sd_test.disk.img
```

smp-mat-sdcard-test

```
truncate -s 4G /tmp/smp-mat-sdcard-test.disk.img  
timeout 2000 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-drive file=/tmp/smp-mat-sdcard-test.disk.img,if=sd,format=raw \  
-kernel $D/smp-mat-sdcard-test-qemu.bin \  
< /dev/null  
rm -f /tmp/smp-mat-sdcard-test.disk.img
```

smp-mat-test

```
timeout 900 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/smp-mat-test-qemu.bin \  
< /dev/null
```

smp-num-test

```
truncate -s 4G /tmp/smp-num-test.disk.img
timeout 1000 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -drive file=/tmp/smp-num-test.disk.img,if=sd,format=raw \
    -kernel $D/smp-num-test-qemu.bin \
    < /dev/null
rm -f /tmp/smp-num-test.disk.img
```

smp-pipeline-test

```
truncate -s 4G /tmp/smp-pipeline-test.disk.img
timeout 1000 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -drive file=/tmp/smp-pipeline-test.disk.img,if=sd,format=raw \
    -kernel $D/smp-pipeline-test-qemu.bin \
    < /dev/null
rm -f /tmp/smp-pipeline-test.disk.img
```

smp-pro-cons-test

```
timeout 1000 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -kernel $D/smp-pro-cons-test-qemu.bin \
    < /dev/null
```

smp_test0

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -kernel $D/smp_test0-qemu.bin \
    < /dev/null
```

smp_test1

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -kernel $D/smp_test1-qemu.bin \
    < /dev/null
```

smp_test2

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/smp_test2-qemu.bin \  
< /dev/null
```

smp_test3

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/smp_test3-qemu.bin \  
< /dev/null
```

smp_test4

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/smp_test4-qemu.bin \  
< /dev/null
```

usb_test

```
timeout 200 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/usb_test-qemu.bin \  
< /dev/null
```

3.6 aarch64-rpi3b — QEMU

```
D=$BUILD/aarch64-rpi3b-cmake-gcc-debug/platform-bin/port-tests/test
```

cmsis-os-validator

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/cmsis-os-validator-qemu.bin \  
< /dev/null
```

mutex-stress

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/mutex-stress-qemu.bin \  
< /dev/null
```

rtos-apis

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/rtos-apis-qemu.bin \  
< /dev/null
```

sd_test

```
python3 $A64/test/rpi3b/sd_test/flatfs_tool.py make-seed /tmp/sd_test.disk.img  
timeout 450 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-drive file=/tmp/sd_test.disk.img,if=sd,format=raw \  
-kernel $D/sd_test-qemu.bin \  
< /dev/null  
rm -f /tmp/sd_test.disk.img
```

smp-mat-sdcard-test

```
truncate -s 4G /tmp/smp-mat-sdcard-test.disk.img  
timeout 2000 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-drive file=/tmp/smp-mat-sdcard-test.disk.img,if=sd,format=raw \  
-kernel $D/smp-mat-sdcard-test-qemu.bin \  
< /dev/null  
rm -f /tmp/smp-mat-sdcard-test.disk.img
```

smp-mat-test

```
timeout 900 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/smp-mat-test-qemu.bin \  
< /dev/null
```

smp-num-test

```
truncate -s 4G /tmp/smp-num-test.disk.img
timeout 1000 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -drive file=/tmp/smp-num-test.disk.img,if=sd,format=raw \
    -kernel $D/smp-num-test-qemu.bin \
    < /dev/null
rm -f /tmp/smp-num-test.disk.img
```

smp-pipeline-test

```
truncate -s 4G /tmp/smp-pipeline-test.disk.img
timeout 1000 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -drive file=/tmp/smp-pipeline-test.disk.img,if=sd,format=raw \
    -kernel $D/smp-pipeline-test-qemu.bin \
    < /dev/null
rm -f /tmp/smp-pipeline-test.disk.img
```

smp-pro-cons-test

```
timeout 1000 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -kernel $D/smp-pro-cons-test-qemu.bin \
    < /dev/null
```

smp_test0

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -kernel $D/smp_test0-qemu.bin \
    < /dev/null
```

smp_test1

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \
    -semihosting-config enable=on,target=native \
    -kernel $D/smp_test1-qemu.bin \
    < /dev/null
```

smp_test2

```
timeout 300 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/smp_test2-qemu.bin \  
< /dev/null
```

smp_test3

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/smp_test3-qemu.bin \  
< /dev/null
```

smp_test4

```
timeout 150 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/smp_test4-qemu.bin \  
< /dev/null
```

usb_test

```
timeout 200 $QEMU -M raspi3b -smp 4 -nographic -serial none \  
-semihosting-config enable=on,target=native \  
-kernel $D/usb_test-qemu.bin \  
< /dev/null
```

4. Hardware

4.1 What a hardware run does

One test at a time: the image is loaded into RAM over whatever was there before, and the Pi has no reset a script can drive, so each test starts with a reset through the watchdog. These commands do not use the UART console; keep your own terminal on it (for example `tio -b 115200 /dev/ttyACM0`). The test's semihosting output appears in the OpenOCD output; the test passed when it prints `RESULT: PASS`. Then stop OpenOCD (Ctrl-C).

The session file is written with a quoted here-document (so that the Tcl `$score` stays literal); the ELF path is then put in with `sed`.

Only core 0 is started at the entry. Cores 1–3 are resumed where they were halted: after the watchdog reset they wait in the firmware's own loop, and the test releases them itself (`release_one()` in `test/boards/rpi-zero-2w/src/smp.cpp`) after it has written their `__smp_spin` slot. So nothing is read from the

ELF and nothing is written to `__smp_spin`. This is what `hw.sh` does (`UOS_HW_RESUME=cpsr-first` / `pc-first` , `UOS_HW_SPIN_WORDS=0`); the previous flow, which started all four cores and zeroed `__smp_spin` , is kept in `hw.sh.bak` .

It needs an SD card whose kernel leaves cores 1–3 in the firmware's loop, booting the same width as the port, because the two ports release the cores differently:

Port	SD card <code>config.txt</code>	Cores 1–3 wait for	After the reset, cores 1–3 are at
AArch32	<code>arm_64bit=0</code>	their mailbox 3	<code>pc=0x7a</code> , Thumb, Hypervisor
AArch64	<code>arm_64bit=1</code> (e.g. <code>\$A64/test/boards/rpi-zero-2w/hw-park/</code>)	their release word at <code>0xd8 + 8*core</code>	<code>pc=0x7c</code> , EL2H

Tested on a Pi Zero 2 W on 2026-10-09: every test below except `usb_test` passed this way, on both ports.

4.2 Raspberry Pi (Zero 2 W and 3 B), both ports

	AArch32	AArch64
OpenOCD config, J-Link (default)	<code>\$A32/test/boards/rpi-zero-2w/openocd-jlink-rpi3.cfg</code>	<code>\$A64/test/boards/rpi-zero-2w/openocd-jlink-rpi3.cfg</code>
OpenOCD config, Olimex ARM-USB-OCD	<code>\$A32/test/boards/rpi-zero-2w/openocd-olimex.cfg</code>	<code>\$A64/test/boards/rpi-zero-2w/openocd-olimex.cfg</code>
<code>-c init</code>	yes (the config is declarative)	yes (the config is declarative)
entry	<code>0x1003c</code>	<code>0x80000</code>
start core 0	set CPSR to <code>0x600001da</code> , then resume at the entry	set PC to the entry, then resume
cores 1–3	<code>resume</code> where they are	<code>resume</code> where they are

The Pi 3 B uses the Zero 2 W's configs (same BCM2837). Each test is three steps:

1. **Reset** through the watchdog (`PM_RSTC` / `PM_WDOG` at `0x3f10001c` and `0x3f100024`), then wait 12 s for the board to boot.
2. **Write the session** (`/tmp/session.tcl`): halt the four cores, enable semihosting on each, load the ELF, start core 0 at the entry, let cores 1–3 go on where they are.
3. **Run OpenOCD** with the config and the session.

The sections below write every test with the J-Link config; for the Olimex probe replace `openocd-jlink-rpi3.cfg` by `openocd-olimex.cfg` .

4.3 aarch32-rpi-zero-2w — hardware

```
D=$BUILD/aarch32-rpi-zero-2w-cmake-gcc-debug/platform-bin/port-tests/test
```

cmsis-os-validator (allow about 300 s)

```
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-  
rpi3.cfg -c init \  
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"  
sleep 12  
cat > /tmp/session.tcl <<'EOF'  
echo "--- stage: halt cores 0 1 2 3 ---"  
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }  
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"  
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }  
echo "--- stage: load_image __ELF__ ---"  
targets bcm2837.cpu0  
load_image __ELF__  
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"  
targets bcm2837.cpu0  
reg cpsr 0x600001da  
resume 0x1003c  
foreach core {1 2 3} {  
    targets [format {bcm2837.cpu%d} $core]  
    resume  
}  
EOF  
sed -i "s|__ELF__|$D/cmsis-os-validator-hwd|g" /tmp/session.tcl  
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-  
rpi3.cfg -c init -f /tmp/session.tcl
```

mutex-stress (allow about 300 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/mutex-stress-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

rtos-apis (allow about 300 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/rtos-apis-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

sd_test (allow about 450 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/sd_test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-mat-sdcard-test (allow about 900 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-mat-sdcard-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-mat-test (allow about 900 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-mat-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-num-test (allow about 600 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-num-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-pipeline-test (allow about 600 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-pipeline-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-pro-cons-test (allow about 600 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-pro-cons-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

`smp_test0` (allow about 120 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test0-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp_test1 (allow about 120 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test1-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp_test2 (allow about 120 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test2-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp_test3 (allow about 120 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test3-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp_test4 (allow about 300 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test4-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

usb_test (allow about 300 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/usb_test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

4.4 aarch32-rpi3b — hardware

```
D=$BUILD/aarch32-rpi3b-cmake-gcc-debug/platform-bin/port-tests/test
```

`ctest` registers no hardware case for `usb_test` on the Pi 3 B; its image is built but not listed here.

`cmsis-os-validator` (allow about 300 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/cmsis-os-validator-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

mutex-stress (allow about 300 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/mutex-stress-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

rtos-apis (allow about 300 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/rtos-apis-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

sd_test (allow about 450 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/sd_test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-mat-sdcard-test (allow about 900 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-mat-sdcard-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-mat-test (allow about 900 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-mat-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-num-test (allow about 600 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-num-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-pipeline-test (allow about 600 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-pipeline-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-pro-cons-test (allow about 600 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-pro-cons-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

`smp_test0` (allow about 120 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test0-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp_test1 (allow about 120 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test1-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp_test2 (allow about 120 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test2-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp_test3 (allow about 120 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test3-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp_test4 (allow about 300 s)

```

$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x1003c ---"
targets bcm2837.cpu0
reg cpsr 0x600001da
resume 0x1003c
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test4-hwd|g" /tmp/session.tcl
$OPENOCD -s $A32/test/boards/rpi-zero-2w -s $SCRIPTS -f $A32/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

4.5 **aarch64-rpi-zero-2w** — hardware

```
D=$BUILD/aarch64-rpi-zero-2w-cmake-gcc-debug/platform-bin/port-tests/test
```

cmsis-os-validator (allow about 300 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/cmsis-os-validator-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

mutex-stress (allow about 300 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/mutex-stress-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

rtos-apis (allow about 300 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/rtos-apis-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

sd_test (allow about 450 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/sd_test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-mat-sdcard-test (allow about 900 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-mat-sdcard-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-mat-test (allow about 900 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-mat-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-num-test (allow about 600 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-num-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-pipeline-test (allow about 600 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-pipeline-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-pro-cons-test (allow about 600 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-pro-cons-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

`smp_test0` (allow about 120 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test0-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

`smp_test1` (allow about 120 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test1-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp_test2 (allow about 120 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test2-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp_test3 (allow about 120 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test3-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp_test4 (allow about 300 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test4-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

usb_test (allow about 300 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/usb_test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

4.6 aarch64-rpi3b — hardware

```
D=$BUILD/aarch64-rpi3b-cmake-gcc-debug/platform-bin/port-tests/test
```

`ctest` registers no hardware case for `usb_test` on the Pi 3 B; its image is built but not listed here.

`cmsis-os-validator` (allow about 300 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/cmsis-os-validator-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

mutex-stress (allow about 300 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/mutex-stress-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

rtos-apis (allow about 300 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/rtos-apis-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

sd_test (allow about 450 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/sd_test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-mat-sdcard-test (allow about 900 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-mat-sdcard-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-mat-test (allow about 900 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-mat-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-num-test (allow about 600 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-num-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-pipeline-test (allow about 600 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-pipeline-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp-pro-cons-test (allow about 600 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp-pro-cons-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

`smp_test0` (allow about 120 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test0-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

`smp_test1` (allow about 120 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test1-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp_test2 (allow about 120 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test2-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp_test3 (allow about 120 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test3-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

smp_test4 (allow about 300 s)

```

$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init \
    -c "targets bcm2837.cpu0; halt; mww 0x3f100024 0x5a000001; mww 0x3f10001c 0x5a000020; shutdown"
sleep 12
cat > /tmp/session.tcl <<'EOF'
echo "--- stage: halt cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
foreach core {0 1 2 3} { targets [format {bcm2837.cpu%d} $core]; arm semihosting enable }
echo "--- stage: load_image __ELF__ ---"
targets bcm2837.cpu0
load_image __ELF__
echo "--- stage: resume cores 0 1 2 3 at 0x80000 ---"
targets bcm2837.cpu0
reg pc 0x80000
echo " start PC core 0 = [reg pc]"
resume
foreach core {1 2 3} {
    targets [format {bcm2837.cpu%d} $core]
    resume
}
EOF
sed -i "s|__ELF__|$D/smp_test4-hwd|g" /tmp/session.tcl
$OPENOCD -s $A64/test/boards/rpi-zero-2w -s $SCRIPTS -f $A64/test/boards/rpi-zero-2w/openocd-jlink-
rpi3.cfg -c init -f /tmp/session.tcl

```

4.7 aarch32-luckfox-lyra — hardware

The Luckfox Lyra (RK3506, three Cortex-A7) is reached over SWD by a CMSIS-DAP probe (`$A32/test/boards/luckfox-lyra/openocd.cfg`), which runs `init` itself). Only core 0 is a debug target at load time — the image releases the other two — so only core 0 is halted, loaded and resumed, and there are no `__smp_spin` words to zero. Before the load, the MMU and caches the miniloader left on are switched off. There is no reset step. The entry read from every Lyra ELF is `0x20003c`.

```

D=$BUILD/aarch32-luckfox-lyra-cmake-gcc-debug/platform-bin/port-tests/test
CFG=$A32/test/boards/luckfox-lyra

```

`mutex-stress-test` (allow about 300 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/mutex-stress-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

sd_test (allow about 450 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/sd_test-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

smp-mat-sdcard-test (allow about 900 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp-mat-sdcard-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

smp-mat-test (allow about 900 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp-mat-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

smp-num-test (allow about 600 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp-num-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

smp-pipeline-test (allow about 600 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp-pipeline-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

smp-pro-cons-test (allow about 600 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [$_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp-pro-cons-test-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

`smp_test0` (allow about 120 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [$_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp_test0-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

smp_test1 (allow about 120 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp_test1-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

smp_test2 (allow about 120 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp_test2-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

smp_test3 (allow about 120 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp_test3-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

smp_test4 (allow about 300 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp_test4-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

smp_test5 (allow about 300 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp_test5-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

smp_test6 (allow about 300 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp_test6-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

smp_test7 (allow about 300 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp_test7-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

`smp_test_int` (allow about 300 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTL 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp_test_int-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

`smp_test_int2` (allow about 300 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp_test_int2-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

`smp_test_int3` (allow about 300 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp_test_int3-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

`smp_test_int4` (allow about 300 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__|$D/smp_test_int4-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

`smp_test_int5` (allow about 300 s)

```

cat > /tmp/session.tcl <<'EOF'
echo "--- stage: JTAG clock -> 4000 kHz ---"
adapter speed 4000
echo "--- stage: halt cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; halt }
echo "--- stage: enable semihosting on cores 0 ---"
foreach core {0} { targets [format {rk3506.a7.%d} $core]; arm semihosting enable }
echo "--- stage: pre-load ---"
targets rk3506.a7.0
if {[catch {
    set _t [target current]
    set _s [_t arm mrc 15 0 1 0 0]
    $_t arm mcr 15 0 1 0 0 [expr {$_s & ~0x1005}]
    $_t arm mcr 15 0 7 5 0 0
    $_t arm mcr 15 0 7 5 6 0
    $_t arm mcr 15 0 8 7 0 0
} _e]} { echo "SANITIZE-ERR: $_e" } else { echo [format "SANITIZE-OK: SCTLR 0x%08x -> MMU/caches
off" $_s] }
echo "--- stage: load_image __ELF__ ---"
targets rk3506.a7.0
load_image __ELF__
echo "--- stage: resume cores 0 at 0x20003c ---"
foreach core {0} {
    targets [format {rk3506.a7.%d} $core]
    resume 0x20003c
}
EOF
sed -i "s|__ELF__$D/smp_test_int5-hwd|g" /tmp/session.tcl
$OPENOCD -s $CFG -s $SCRIPTS -f $CFG/openocd.cfg -f /tmp/session.tcl

```

5. Cortex-M on QEMU

5.1 The commands

These tests are started by `ctest` directly (no runner script). A test passed when QEMU exits with status 0; the suites end with `Hasta la Vista!`, the port tests print `RESULT: PASS`. `ctest` gives each 1200 s (suites) or 300 s (port tests).

Checked on 2026-10-09 with these exact commands: `2xcortex-m33` `fp-switch-test` and `pico2-1cpu` `mutex-stress-test` (exit 0), `cortexm-pico2` `smp-test1` (`RESULT: PASS` , exit 0).

5.2 2xcortex-m33 — QEMU

The harness suites on two Cortex-M33 (SSE-200) on QEMU's `mps2-an521`.

```
D=$BUILD/2xcortex-m33-cmake-gcc-debug/platform-bin
```

rtos-apis-test

```
timeout 1200 $QEMUARM --machine mps2-an521 --cpu cortex-m33 --smp 2 \  
  --global sse-200.CPU0_FPU=on --global sse-200.CPU0_DSP=on \  
  --global sse-200.CPU1_FPU=on --global sse-200.CPU1_DSP=on \  
  --kernel $D/rtos-apis-test.elf --nographic -d unimp,guest_errors \  
  --semihosting-config enable=on,target=native < /dev/null
```

mutex-stress-test

```
timeout 1200 $QEMUARM --machine mps2-an521 --cpu cortex-m33 --smp 2 \  
  --global sse-200.CPU0_FPU=on --global sse-200.CPU0_DSP=on \  
  --global sse-200.CPU1_FPU=on --global sse-200.CPU1_DSP=on \  
  --kernel $D/mutex-stress-test.elf --nographic -d unimp,guest_errors \  
  --semihosting-config enable=on,target=native < /dev/null
```

fp-switch-test

```
timeout 1200 $QEMUARM --machine mps2-an521 --cpu cortex-m33 --smp 2 \  
  --global sse-200.CPU0_FPU=on --global sse-200.CPU0_DSP=on \  
  --global sse-200.CPU1_FPU=on --global sse-200.CPU1_DSP=on \  
  --kernel $D/fp-switch-test.elf --nographic -d unimp,guest_errors \  
  --semihosting-config enable=on,target=native < /dev/null
```

cmsis-os-validator-test

```
timeout 1200 $QEMUARM --machine mps2-an521 --cpu cortex-m33 --smp 2 \  
  --global sse-200.CPU0_FPU=on --global sse-200.CPU0_DSP=on \  
  --global sse-200.CPU1_FPU=on --global sse-200.CPU1_DSP=on \  
  --kernel $D/cmsis-os-validator-test.elf --nographic -d unimp,guest_errors \  
  --semihosting-config enable=on,target=native < /dev/null
```

5.3 pico2-1cpu — QEMU

The harness suites on one Cortex-M33 on QEMU's `mps2-an505`.

```
D=$BUILD/pico2-1cpu-cmake-gcc-debug/platform-bin
```

rtos-apis-test

```
timeout 1200 $QEMUARM --machine mps2-an505 --cpu cortex-m33 \  
--kernel $D/rtos-apis-test.elf --nographic -d unimp,guest_errors \  
--semihosting-config enable=on,target=native < /dev/null
```

mutex-stress-test

```
timeout 1200 $QEMUARM --machine mps2-an505 --cpu cortex-m33 \  
--kernel $D/mutex-stress-test.elf --nographic -d unimp,guest_errors \  
--semihosting-config enable=on,target=native < /dev/null
```

fp-switch-test

```
timeout 1200 $QEMUARM --machine mps2-an505 --cpu cortex-m33 \  
--kernel $D/fp-switch-test.elf --nographic -d unimp,guest_errors \  
--semihosting-config enable=on,target=native < /dev/null
```

cmsis-os-validator-test

```
timeout 1200 $QEMUARM --machine mps2-an505 --cpu cortex-m33 \  
--kernel $D/cmsis-os-validator-test.elf --nographic -d unimp,guest_errors \  
--semihosting-config enable=on,target=native < /dev/null
```

5.4 cortexm-pico2 — QEMU

The port tests that have a QEMU build run on a generic Cortex-M7 (QEMU's `mps2-an500`).

```
D=$BUILD/cortexm-pico2-cmake-gcc-debug/platform-bin/port-tests/test
```

cmsis-os-validator

```
timeout 300 $QEMUARM -M mps2-an500 -cpu cortex-m7 \  
-kernel $D/cmsis-os-validator-qemu.elf --nographic -d unimp,guest_errors \  
--semihosting-config enable=on,target=native < /dev/null
```

fp-switch

```
timeout 300 $QEMUARM -M mps2-an500 -cpu cortex-m7 \  
-kernel $D/fp-switch-qemu.elf --nographic -d unimp,guest_errors \  
--semihosting-config enable=on,target=native < /dev/null
```

mutex-stress

```
timeout 300 $QEMUARM -M mps2-an500 -cpu cortex-m7 \  
-kernel $D/mutex-stress-qemu.elf --nographic -d unimp,guest_errors \  
--semihosting-config enable=on,target=native < /dev/null
```

rtos-apis

```
timeout 300 $QEMUARM -M mps2-an500 -cpu cortex-m7 \  
-kernel $D/rtos-apis-qemu.elf --nographic -d unimp,guest_errors \  
--semihosting-config enable=on,target=native < /dev/null
```

sc-test-ko

```
timeout 300 $QEMUARM -M mps2-an500 -cpu cortex-m7 \  
-kernel $D/sc-test-ko-qemu.elf --nographic -d unimp,guest_errors \  
--semihosting-config enable=on,target=native < /dev/null
```

smp-test1

```
timeout 300 $QEMUARM -M mps2-an500 -cpu cortex-m7 \  
-kernel $D/smp-test1-qemu.elf --nographic -d unimp,guest_errors \  
--semihosting-config enable=on,target=native < /dev/null
```

5.5 cortexm-pico2-rp2350b-psram — QEMU

The port tests that have a QEMU build run on a generic Cortex-M7 (QEMU's `mps2-an500`).

```
D=$BUILD/cortexm-pico2-rp2350b-psram-cmake-gcc-debug/platform-bin/port-tests/test
```

fp-switch

```
timeout 300 $QEMUARM -M mps2-an500 -cpu cortex-m7 \  
-kernel $D/fp-switch-qemu.elf --nographic -d unimp,guest_errors \  
--semihosting-config enable=on,target=native < /dev/null
```

sc-test-ko

```
timeout 300 $QEMUARM -M mps2-an500 -cpu cortex-m7 \
  -kernel $D/sc-test-ko-qemu.elf --nographic -d unimp,guest_errors \
  --semihosting-config enable=on,target=native < /dev/null
```

`smp-test1`

```
timeout 300 $QEMUARM -M mps2-an500 -cpu cortex-m7 \
  -kernel $D/smp-test1-qemu.elf --nographic -d unimp,guest_errors \
  --semihosting-config enable=on,target=native < /dev/null
```

6. RP2350 hardware (Pico 2, WeAct RP2350B, Pi-Zero RP2350B)

6.1 What a hardware run does

OpenOCD over a CMSIS-DAP probe. The test image is written to the flash (`program ... verify`), the chip is reset with `reset init` (a bare `reset` does not re-run the bootrom/XIP setup), semihosting is enabled on both cores, and core 1 is resumed **before** core 0 (a debug-halted core 1 does not answer the core-1 launch). The test's console is the board's serial port; its `RESULT:` line is also written through semihosting, so it appears in the OpenOCD output. When `RESULT: PASS` (or `RESULT: FAIL`) appears, stop OpenOCD (Ctrl-C).

One test per power cycle: power-cycle the board before the next test.

Each test is two commands: stop any OpenOCD already attached to that board's config, then run OpenOCD.

6.2 The boards

Platform	OpenOCD config	Probe	Flash
<code>cortexm-pico2</code>	<code>\$C/test/boards/pico2/openocd.cfg</code>	CMSIS-DAP (any)	4 MB
<code>cortexm-pico2-rp2350b-psram</code>	<code>\$C/test/boards/pico2-rp2350b-psram/openocd.cfg</code>	CMSIS-DAP <code>0xc251:0xf001</code>	16 MB
<code>cortexm-pico2-pizero</code>	<code>\$C/test/boards/pico2-pizero/openocd.cfg</code>	CMSIS-DAP <code>0x0416:0x5951</code>	16 MB

6.3 `cortexm-pico2` — hardware, programmed into flash

```
D=$BUILD/cortexm-pico2-cmake-gcc-debug/platform-bin/port-tests/test
```

cmsis-os-validator

```
pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/cmsis-os-validator-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

exc-test

```
pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/exc-test-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

fp-switch

```

pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/fp-switch-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

mutex-stress

```

pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/mutex-stress-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

rtos-apis

```

pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/rtos-apis-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

sc-test-ko

```
pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/sc-test-ko-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

smp-mat-test

```
pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-mat-test-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

smp-test-ko

```

pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test-ko-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test-usb-cdc-acm

```

pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test-usb-cdc-acm-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test-usb-hid

```

pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test-usb-hid-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test0

```
pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test0-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

smp-test1

```
pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test1-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

smp-test2

```

pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test2-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test3

```

pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test3-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test4

```

pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test4-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test5

```
pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test5-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

6.4 cortexm-pico2 — hardware, loaded into RAM

These four tests have their own runner (`$C/test/pico2/<test>/hw.sh`, which the board's `hw.sh` hands them to): the image is loaded into SRAM without writing the flash, the vector table is moved to `0x20000000`, and core 0 is started from the image's initial stack pointer and reset vector.

cmsis-os-validator-ram

```
pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "rp2350.cm0 configure -work-area-phys 0x20070000 -work-area-size 0x8000" \
  -c "load_image \"$D/cmsis-os-validator-ram-hwd\"" \
  -c "verify_image \"$D/cmsis-os-validator-ram-hwd\"" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "set v [read_memory 0x20000000 32 2]" \
  -c "mww 0xe000ed08 0x20000000" \
  -c "reg sp [lindex \v 0]" \
  -c "reg pc [expr {[lindex \v 1] & ~1}]" \
  -c "resume"
```

mutex-stress-ram

```

pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "rp2350.cm0 configure -work-area-phys 0x20070000 -work-area-size 0x8000" \
  -c "load_image \"$D/mutex-stress-ram-hwd\"" \
  -c "verify_image \"$D/mutex-stress-ram-hwd\"" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "set v [read_memory 0x20000000 32 2]" \
  -c "mww 0xe000ed08 0x20000000" \
  -c "reg sp [lindex \"$v 0]" \
  -c "reg pc [expr {[lindex \"$v 1] & ~1}]" \
  -c "resume"

```

rtos-apis-ram

```

pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "rp2350.cm0 configure -work-area-phys 0x20070000 -work-area-size 0x8000" \
  -c "load_image \"$D/rtos-apis-ram-hwd\"" \
  -c "verify_image \"$D/rtos-apis-ram-hwd\"" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "set v [read_memory 0x20000000 32 2]" \
  -c "mww 0xe000ed08 0x20000000" \
  -c "reg sp [lindex \"$v 0]" \
  -c "reg pc [expr {[lindex \"$v 1] & ~1}]" \
  -c "resume"

```

smp-mat-test-ram

```

pkill -9 -f "openocd.*boards/pico2/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "rp2350.cm0 configure -work-area-phys 0x20070000 -work-area-size 0x8000" \
  -c "load_image \"$D/smp-mat-test-ram-hwd\"" \
  -c "verify_image \"$D/smp-mat-test-ram-hwd\"" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "set v [read_memory 0x20000000 32 2]" \
  -c "mww 0xe000ed08 0x20000000" \
  -c "reg sp [lindex $v 0]" \
  -c "reg pc [expr {[lindex $v 1] & ~1}]" \
  -c "resume"

```

6.5 cortexm-pico2-rp2350b-psram — hardware

```
D=$BUILD/cortexm-pico2-rp2350b-psram-cmake-gcc-debug/platform-bin/port-tests/test
```

exc-test

```

pkill -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/exc-test-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

fp-switch

```

pkill -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/fp-switch-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

sc-test-ko

```

pkill -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/sc-test-ko-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-mat-test

```

pkill -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-mat-test-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test-ko

```
pkill -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test-ko-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

smp-test-nested

```
pkill -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test-nested-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

smp-test-nested-clock

```

pkill -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test-nested-clock-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test-nested-clock_200

```

pkill -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test-nested-clock_200-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test-nested-clock_250

```

pkill -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test-nested-clock_250-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test0

```
killall -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test0-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

smp-test1

```
killall -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test1-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

smp-test2

```

pkill -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test2-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test3

```

pkill -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test3-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test4

```

pkill -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test4-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test5

```
pkill -9 -f "openocd.*boards/pico2-rp2350b-psram/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-rp2350b-psram/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test5-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

6.6 cortexm-pico2-pizero — hardware

```
D=$BUILD/cortexm-pico2-pizero-cmake-gcc-debug/platform-bin/port-tests/test
```

exc-test

```
pkill -9 -f "openocd.*boards/pico2-pizero/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-pizero/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/exc-test-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

psram-exec

```

pkill -9 -f "openocd.*boards/pico2-pizero/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-pizero/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/psram-exec-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

psram-mat-test-250

```

pkill -9 -f "openocd.*boards/pico2-pizero/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-pizero/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/psram-mat-test-250-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

sc-test-ko

```

pkill -9 -f "openocd.*boards/pico2-pizero/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-pizero/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/sc-test-ko-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-mat-test

```
pkill -9 -f "openocd.*boards/pico2-pizero/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-pizero/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-mat-test-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

smp-test-ko

```
pkill -9 -f "openocd.*boards/pico2-pizero/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-pizero/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test-ko-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

smp-test-usb-cdc-acm

```

pkill -9 -f "openocd.*boards/pico2-pizero/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-pizero/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test-usb-cdc-acm-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test-usb-hid

```

pkill -9 -f "openocd.*boards/pico2-pizero/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-pizero/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test-usb-hid-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test0

```

pkill -9 -f "openocd.*boards/pico2-pizero/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-pizero/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test0-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"

```

smp-test1

```
killall -9 -f "openocd.*boards/pico2-pizero/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-pizero/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test1-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

smp-test2

```
killall -9 -f "openocd.*boards/pico2-pizero/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-pizero/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test2-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

smp-test3

```
pkill -9 -f "openocd.*boards/pico2-pizero/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-pizero/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test3-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

smp-test4

```
pkill -9 -f "openocd.*boards/pico2-pizero/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-pizero/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test4-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

smp-test5

```
pkill -9 -f "openocd.*boards/pico2-pizero/openocd.cfg"
$OPENOCD -s $SCRIPTS -f $C/test/boards/pico2-pizero/openocd.cfg \
  -c "init" \
  -c "reset init" \
  -c "program \"$D/smp-test5-hwd\" verify" \
  -c "reset init" \
  -c "targets rp2350.cm1" \
  -c "arm semihosting enable" \
  -c "resume" \
  -c "targets rp2350.cm0" \
  -c "arm semihosting enable" \
  -c "resume"
```

7. STM32F4 hardware (Nucleo-F411RE, WeAct F411CE, WeAct F412RE)

7.1 What a hardware run does

OpenOCD halts the chip, writes the test image to the flash, enables semihosting (the trace output is a semihosting call, which faults when no debugger listens), then resets the chip into the image (`reset run`). The semihosting output and the `RESULT:` line appear in the OpenOCD output; the board's UART console (115200 8N1) is separate. When `RESULT: PASS` (or `RESULT: FAIL`) appears, stop OpenOCD (Ctrl-C).

Platform	OpenOCD config	Probe
<code>cortexm-nucleof411</code>	<code>\$C/test/boards/nucleof411/openocd.cfg</code>	the Nucleo's ST-Link (<code>board/st_nucleo_f4.cfg</code> , connect under reset)
<code>cortexm-weactf411</code>	<code>\$C/test/boards/weactf411/openocd.cfg</code>	CMSIS-DAP <code>0x0d28:0x0204</code> (another one: add <code>-c 'set CMSIS_DAP_VID_PID {0x1a86 0x8011}'</code> before <code>-f</code>)
<code>cortexm-weactf412</code>	<code>\$C/test/boards/weactf412/openocd.cfg</code>	ST-Link (V2 / V2-1 / V3)

The SWD clock of the two WeAct configs is 1000 kHz; another one is set with `-c 'set ADAPTER_KHZ <n>'` before `-f` .

7.2 `cortexm-nucleof411` — hardware

```
D=$BUILD/cortexm-nucleof411-cmake-gcc-debug/platform-bin/port-tests/test
```

`cmsis-os-validator`

```
$OPENOCD -s $SCRIPTS -f $C/test/boards/nucleof411/openocd.cfg \  
-c "init" \  
-c "reset halt" \  
-c "program \"$D/cmsis-os-validator-hwd\"" \  
-c "arm semihosting enable" \  
-c "reset run"
```

`mos-test1`

```
$OPENOCD -s $SCRIPTS -f $C/test/boards/nucleof411/openocd.cfg \  
-c "init" \  
-c "reset halt" \  
-c "program \"$D/mos-test1-hwd\"" \  
-c "arm semihosting enable" \  
-c "reset run"
```

mutex-stress

```
$OPENOCD -s $SCRIPTS -f $C/test/boards/nucleof411/openocd.cfg \  
-c "init" \  
-c "reset halt" \  
-c "program \"$D/mutex-stress-hwd\"" \  
-c "arm semihosting enable" \  
-c "reset run"
```

rtos-apis

```
$OPENOCD -s $SCRIPTS -f $C/test/boards/nucleof411/openocd.cfg \  
-c "init" \  
-c "reset halt" \  
-c "program \"$D/rtos-apis-hwd\"" \  
-c "arm semihosting enable" \  
-c "reset run"
```

7.3 cortexm-weactf411 — hardware

```
D=$BUILD/cortexm-weactf411-cmake-gcc-debug/platform-bin/port-tests/test
```

cmsis-os-validator

```
$OPENOCD -s $SCRIPTS -f $C/test/boards/weactf411/openocd.cfg \  
-c "init" \  
-c "reset halt" \  
-c "program \"$D/cmsis-os-validator-hwd\"" \  
-c "arm semihosting enable" \  
-c "reset run"
```

mos-test1

```
$OPENOCD -s $SCRIPTS -f $C/test/boards/weactf411/openocd.cfg \  
-c "init" \  
-c "reset halt" \  
-c "program \"$D/mos-test1-hwd\"" \  
-c "arm semihosting enable" \  
-c "reset run"
```

mutex-stress

```
$OPENOCD -s $SCRIPTS -f $C/test/boards/weactf411/openocd.cfg \  
-c "init" \  
-c "reset halt" \  
-c "program \"$D/mutex-stress-hwd\"" \  
-c "arm semihosting enable" \  
-c "reset run"
```

rtos-apis

```
$OPENOCD -s $SCRIPTS -f $C/test/boards/weactf411/openocd.cfg \  
-c "init" \  
-c "reset halt" \  
-c "program \"$D/rtos-apis-hwd\"" \  
-c "arm semihosting enable" \  
-c "reset run"
```

spi-pipeline

```
$OPENOCD -s $SCRIPTS -f $C/test/boards/weactf411/openocd.cfg \  
-c "init" \  
-c "reset halt" \  
-c "program \"$D/spi-pipeline-hwd\"" \  
-c "arm semihosting enable" \  
-c "reset run"
```

7.4 cortexm-weactf412 — hardware

```
D=$BUILD/cortexm-weactf412-cmake-gcc-debug/platform-bin/port-tests/test
```

cmsis-os-validator

```
$OPENOCD -s $SCRIPTS -f $C/test/boards/weactf412/openocd.cfg \  
-c "init" \  
-c "reset halt" \  
-c "program \"$D/cmsis-os-validator-hwd\"" \  
-c "arm semihosting enable" \  
-c "reset run"
```

mos-test1

```
$OPENOCD -s $SCRIPTS -f $C/test/boards/weactf412/openocd.cfg \  
-c "init" \  
-c "reset halt" \  
-c "program \"$D/mos-test1-hwd\"" \  
-c "arm semihosting enable" \  
-c "reset run"
```

mutex-stress

```
$OPENOCD -s $SCRIPTS -f $C/test/boards/weactf412/openocd.cfg \  
-c "init" \  
-c "reset halt" \  
-c "program \"$D/mutex-stress-hwd\"" \  
-c "arm semihosting enable" \  
-c "reset run"
```

rtos-apis

```
$OPENOCD -s $SCRIPTS -f $C/test/boards/weactf412/openocd.cfg \  
-c "init" \  
-c "reset halt" \  
-c "program \"$D/rtos-apis-hwd\"" \  
-c "arm semihosting enable" \  
-c "reset run"
```

uart-test1

```
$OPENOCD -s $SCRIPTS -f $C/test/boards/weactf412/openocd.cfg \  
-c "init" \  
-c "reset halt" \  
-c "program \"$D/uart-test1-hwd\"" \  
-c "arm semihosting enable" \  
-c "reset run"
```

The legacy single-core platforms `nucleo-f411re`, `nucleo-f767zi` and `nucleo-h743zi` are not here: `xpack-dev-smp.md` §21.3 records that they fail at configure.

8. POSIX native (the host, `native-cmake-sys`)

8.1 The commands

The port tests are host executables; `test_smp/run-host.sh` starts each one through `/usr/bin/env` under `timeout`, giving the tests that use a card the path of a disk image file in `UOS_SD_IMAGE`. A test passed when it prints `RESULT: PASS`. The CMSIS-RTOS validator is started by `ctest` directly and passes when it exits with status 0 (`Test Result: PASSED`).

Checked on 2026-10-09 with these exact commands: `smp_test0` (`RESULT: PASS (10 heartbeats on core 0)`) and `flatfs-test` with its image (`RESULT: PASS`), both exit 0.

```
D=$BUILD/native-cmake-sys-debug/platform-bin/port-tests
mkdir -p $D/.host-logs
```

8.2 The tests

`flatfs-test`

```
timeout 300 /usr/bin/env UOS_SD_IMAGE=$D/.host-logs/flatfs-test.disk.img $D/flatfs-test-host
```

`mutex-ceiling-test`

```
timeout 300 /usr/bin/env $D/mutex-ceiling-test-host
```

`mutex-stress`

```
timeout 300 /usr/bin/env $D/mutex-stress-host
```

`rtos-apis`

```
timeout 300 /usr/bin/env $D/rtos-apis-host
```

`smp-mat-test`

```
timeout 900 /usr/bin/env $D/smp-mat-test-host
```

`smp-mutex-stress`

```
timeout 300 /usr/bin/env $D/smp-mutex-stress-host
```

smp-num-test

```
timeout 1000 /usr/bin/env UOS_SD_IMAGE=$D/.host-logs/smp-num-test.disk.img $D/smp-num-test-host
```

smp-pipeline-test

```
timeout 1000 /usr/bin/env UOS_SD_IMAGE=$D/.host-logs/smp-pipeline-test.disk.img $D/smp-pipeline-test-host
```

smp-pro-cons-test

```
timeout 1000 /usr/bin/env $D/smp-pro-cons-test-host
```

smp-rtos-apis

```
timeout 300 /usr/bin/env $D/smp-rtos-apis-host
```

smp_test0

```
timeout 150 /usr/bin/env $D/smp_test0-host
```

smp_test1

```
timeout 150 /usr/bin/env $D/smp_test1-host
```

smp_test2

```
timeout 300 /usr/bin/env $D/smp_test2-host
```

smp_test3

```
timeout 150 /usr/bin/env $D/smp_test3-host
```

smp_test4

```
timeout 150 /usr/bin/env $D/smp_test4-host
```

cmsis-os-validator

```
cd $BUILD/native-cmake-sys-debug/platform-bin
timeout 600 ./cmsis-os-validator-test
```

9. The same tests through `ctest`

The commands above are what these run:

```
cd $BUILD/<platform>-cmake-gcc-debug
ctest -N                                # the test names
ctest -R '^<platform>-<test>-qemu$' -V   # one QEMU test (run-qemu.sh)
ctest -L qemu -V                        # all the QEMU tests
ctest -R '^<platform>-<test>-hwd$' -V    # one hardware test (the board's hw.sh)
```

For the native platform the names are `native-<test>-host`; for the Cortex-M QEMU suites, `<platform>-<suite>-test`. The Cortex-M boards' `hw.sh` run OpenOCD themselves (sections 6 and 7); `ctest` gives them 240 s (`cortexm-pico2`) or 600 s (the others).

The AArch32/AArch64 board `hw.sh` looks for the shared runner in `micro-os-plus-iii-smp` (or `micro-os-plus-iii-smp.git`) next to the port. When the kernel folder is named `micro-os-plus-iii`, as here, it fails with `.../micro-os-plus-iii-smp.git/test_smpl/run-hw.sh: No such file or directory` unless `UOS_SMP_DIR` is set:

```
UOS_SMP_DIR=$K ctest -R '^aarch32-rpi3b-smp_test0-hwd$' -V
```

`ctest` calls `hw.sh <test> 600`, so every hardware case gets 600 s; the per-test times in section 4 are the ones `run-hw.sh` uses when no time is given.